```ts
/**
 * Sarvam AI — Speech-to-Text
 *
 * Batch:     POST https://api.sarvam.ai/speech-to-text
 * Streaming: wss://api.sarvam.ai/speech-to-text/ws
 *
 * Official docs:
 *   https://docs.sarvam.ai/api-reference-docs/speech-to-text/apis/streaming
 *
 * ■■ WebSocket connection parameters (query string) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 *   model               "saaras:v3" (recommended) | "saarika:v2.5" (legacy)
 *   language_code       BCP-47, e.g. "ta-IN", "hi-IN", "en-IN"
 *   mode                "transcribe" | "codemix" | "verbatim" | "translit" | "translate"
 *                       (saaras:v3 only; translate = speech-to-English)
 *   sample_rate         16000 (default) | 8000
 *   input_audio_codec   "pcm_s16le" | "wav"
 *   high_vad_sensitivity true → silence boundary = 2 VAD frames (~64ms @16kHz, ~128ms @8kHz);
 *                        default (false) is 18-of-24 frames ≈ 1s+. Fine-tune params
 *                        (negative_frames_count etc.) can override — see Sarvam Fine VAD Tuning docs.
 *   vad_signals         true → emits START_SPEECH / END_SPEECH events
 *
 * ■■ Auth ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 *   Header: api-subscription-key: <key>
 *
 * ■■ Audio message (JSON text, sent as each PCM chunk arrives) ■■■■■■■■■■■■■■■■■■
 *   { audio: { data: "<base64 raw PCM>", encoding: "audio/wav", sample_rate: 16000 } }
 *   NOTE: encoding enum MUST be "audio/wav" — "pcm_s16le" is rejected by Sarvam's
 *   Pydantic model even though the URL param `input_audio_codec=pcm_s16le` sets
 *   the actual codec. Confirmed via live WebSocket probe (June 2026).
 *
 * ■■ Server → client messages (JSON text) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 *   { type: "events", data: { signal_type: "START_SPEECH" | "END_SPEECH" } }
 *   { type: "data",   data: { transcript: string } }
 *
 * ■■ Close codes ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 *   1000 Normal closure (intentional)
 *   1001 Going away        — reconnect
 *   1006 Abnormal (no close frame) — reconnect with backoff
 *   1011 Server error      — reconnect with backoff
 *   4xxx Auth / quota      — do NOT retry; fix first
 */

import { WebSocket } from "ws";
import { pcmToWav } from "../../audio.js";
import type { STTAdapter, SttFinalMetadata } from "./base.js";
import type { STTConfig, StreamSTT } from "../../types.js";
import { logger } from "../../../lib/logger.js";
import { resolveSarvamSttMode, type SarvamSttMode } from "./sarvam-mode.js";

export { resolveSarvamSttMode } from "./sarvam-mode.js";

function envInt(name: string, def: number): number {
  const v = process.env[name];
  if (!v) return def;
  const n = parseInt(v, 10);
  return isNaN(n) ? def : n;
}

const BATCH_URL = "https://api.sarvam.ai/speech-to-text";
const WS_BASE   = "wss://api.sarvam.ai/speech-to-text/ws";

/** Only saaras:v3 supports the `mode` parameter. */
const MODE_CAPABLE = new Set(["saaras:v3"]);

/**
 * Strip leaked diarization/speaker labels (e.g. "Speaker 1:", "speaker:") from a
 * transcript line. Some Sarvam responses prefix utterances with a speaker tag that
 * would otherwise leak verbatim into the LLM context (call 548). Removes the label
 * at the start of the string or immediately after any newline, then trims.
 */
function stripDiarizationLabels(text: string): string {
  return text.replace(/(^|\n)\s*speaker\s*\d*\s*:\s*/gi, "$1").trim();
}
```

```
74
75  /** BCP-47 codes accepted by EVERY Sarvam STT model (saarika:v2.5's 11-language
76   *  set: 10 Indian + English). Note: Odia uses od-IN, not or-IN. */
77  const CORE_LANG = new Set([
78    "bn-IN", "en-IN", "gu-IN", "hi-IN", "kn-IN",
79    "ml-IN", "mr-IN", "od-IN", "pa-IN", "ta-IN", "te-IN",
80  ]);
81
82  /** Languages supported ONLY by saaras:v3 (the 23-language model — 22 Indian +
83   *  English): Assamese, Bodo, Dogri, Konkani, Kashmiri, Maithili, Manipuri,
84   *  Nepali, Sanskrit, Santali, Sindhi, Urdu. Previously these were rejected and
85   *  silently fell back to en-IN even on saaras:v3. */
86  const SAARAS_EXTRA_LANG = new Set([
87    "as-IN", "brx-IN", "doi-IN", "kok-IN", "ks-IN", "mai-IN",
88    "mni-IN", "ne-IN", "sa-IN", "sat-IN", "sd-IN", "ur-IN",
89  ]);
90
91  /** ISO-639 bare codes → Sarvam BCP-47. */
92  const LANG_MAP: Record<string, string> = {
93    bn: "bn-IN", en: "en-IN", gu: "gu-IN", hi: "hi-IN",
94    kn: "kn-IN", ml: "ml-IN", mr: "mr-IN",
95    or: "od-IN", od: "od-IN",             // Odia: BCP-47 is or-IN but Sarvam uses od-IN
96    pa: "pa-IN", ta: "ta-IN", te: "te-IN",
97    // saaras:v3-only languages
98    as: "as-IN", brx: "brx-IN", doi: "doi-IN", kok: "kok-IN", ks: "ks-IN",
99    mai: "mai-IN", mni: "mni-IN", ne: "ne-IN", sa: "sa-IN", sat: "sat-IN",
100   sd: "sd-IN", ur: "ur-IN",
101 };
102
103 function normaliseLang(code: string | null | undefined, model: string): string {
104   if (!code) return "en-IN";
105   // saaras models cover the full 23-language set; saarika only the 11-language core.
106   const isValid = (c: string) =>
107     CORE_LANG.has(c) || (model.startsWith("saaras") && SAARAS_EXTRA_LANG.has(c));
108   const t = code.trim();
109   if (isValid(t)) return t;
110   const bare = t.split("-")[0]?.toLowerCase() ?? "";
111   const mapped = LANG_MAP[bare];
112   if (mapped && isValid(mapped)) return mapped;
113   logger.warn({ rawLang: code, model }, "Sarvam STT: unsupported language for this model — falling b
    ack to en-IN");
114   return "en-IN";
115 }
116
117 interface SarvamMsg {
118   type: string;
119   /** Sarvam attaches a request_id to responses — quoting it is the fastest path for support escalat
    ions. */
120   request_id?: string;
121   data?: { transcript?: string; signal_type?: string; message?: string; language_code?: string; requ
    est_id?: string };
122 }
123
124 export class SarvamSTT implements STTAdapter {
125   private readonly key: string;
126   private readonly model: string;
127   private readonly langCode: string;
128   private readonly mode: SarvamSttMode;
129   private readonly explicitMode: string | null;
130   private readonly highVadSensitivity: boolean;
131   private readonly speechEndFlushMs: number;
132   private readonly incompleteHoldMs: number;
133   private readonly cfg: STTConfig;
134   /** Native sample rate to actually stream to Sarvam — 8000 for telephony, 16000 for webcall. */
135   private readonly sampleRate: 8000 | 16000;
136
137   constructor(
138     apiKey: string,
139     model: string | null,
140     extraConfig: string | null,
141     cfg: STTConfig,
142   ) {
143     this.key = apiKey;
144     this.cfg = cfg;
```

```
    this.sampleRate = cfg.nativeSampleRate === 8000 ? 8000 : 16000;

    const extra = extraConfig
      ? (JSON.parse(extraConfig) as {
          mode?: string;
          highVadSensitivity?: boolean;
          speechEndFlushMs?: number;
          incompleteHoldMs?: number;
        })
      : {};

    // Model: saaras:v3 (recommended) or saarika:v2.5 (legacy) — the ONLY two
    // models the streaming WS endpoint accepts (live-verified). "saaras:v2.5"
    // was never a valid streaming model; normalise it to saaras:v3 so a stale
    // stored value can't 400 the socket into a reconnect loop.
    const requested = (model ?? "").trim();
    this.model = ["saaras:v3", "saarika:v2.5"].includes(requested)
      ? requested
      : "saaras:v3";

    this.langCode = normaliseLang(cfg.language, this.model);
    this.explicitMode = extra.mode ?? null;

    // Mode: explicit agent config wins; else language-aware default (en-IN →
    // transcribe to avoid codemix writing English speech in Tamil script).
    this.mode = MODE_CAPABLE.has(this.model)
      ? resolveSarvamSttMode(this.langCode, this.explicitMode)
      : "transcribe";

    this.highVadSensitivity = typeof extra.highVadSensitivity === "boolean"
      ? extra.highVadSensitivity
      : true;
    this.speechEndFlushMs = typeof extra.speechEndFlushMs === "number" && Number.isFinite(extra.speechEndFlushMs)
      ? Math.max(0, Math.min(200, Math.round(extra.speechEndFlushMs)))
      : 30;
    // Prefer agent UI value; env SARVAM_COALESCE_INCOMPLETE_MS remains a deploy-wide override.
    const envIncomplete = envInt("SARVAM_COALESCE_INCOMPLETE_MS", -1);
    this.incompleteHoldMs = envIncomplete >= 0
      ? Math.max(50, Math.min(1000, envIncomplete))
      : (typeof extra.incompleteHoldMs === "number" && Number.isFinite(extra.incompleteHoldMs)
        ? Math.max(50, Math.min(1000, Math.round(extra.incompleteHoldMs)))
        : 220);

    logger.info(
      {
        langCode: this.langCode,
        mode: this.mode,
        model: this.model,
        explicitMode: this.explicitMode,
        highVadSensitivity: this.highVadSensitivity,
        speechEndFlushMs: this.speechEndFlushMs,
        incompleteHoldMs: this.incompleteHoldMs,
      },
      "Sarvam STT: mode resolved",
    );
  }

  // ── Batch REST ──────────────────────────────────────────────────────────

  async transcribe(audioBuffer: Buffer, sampleRate = this.sampleRate): Promise<string> {
    const wav = pcmToWav(audioBuffer, sampleRate);

    const formData = new FormData();
    // Copy into a plain ArrayBuffer to avoid SharedArrayBuffer incompatibility with Blob.
    const wavBuf = wav.buffer.slice(wav.byteOffset, wav.byteOffset + wav.byteLength) as ArrayBuffer;

    formData.append("file", new Blob([wavBuf], { type: "audio/wav" }), "audio.wav");
    formData.append("model", this.model === "saaras:v3" ? "saaras:v3" : "saarika:v2.5");
    formData.append("language_code", this.langCode);
    if (MODE_CAPABLE.has(this.model)) {
      formData.append("mode", this.mode);
    }

```

```
    const resp = await fetch(BATCH_URL, {
      method: "POST",
      headers: { "api-subscription-key": this.key },
      body: formData,
    });

    if (!resp.ok) {
      const err = await resp.text();
      throw new Error(`Sarvam STT batch error ${resp.status}: ${err}`);
    }

    const json = (await resp.json()) as { transcript?: string };
    return stripDiarizationLabels(json.transcript ?? "");
  }

  // ── Streaming WebSocket ──────────────────────────────────────────────────

  transcribeStream(
    onTranscript: (
      text: string,
      isFinal: boolean,
      detectedLanguage?: string,
      metadata?: SttFinalMetadata,
    ) => void,
    onClose?: () => void,
  ): StreamSTT {
    // Capture class fields as locals so closures (plain object methods, nested
    // functions) can reference them without relying on `this` binding.
    // langCode is MUTABLE: setLanguage() live-switches it mid-call (explicit
    // caller language request) and reconnects the socket with the new hint.
    let   langCode   = this.langCode;
    let   mode       = this.mode;
    const explicitMode = this.explicitMode;
    const model      = this.model;
    const key        = this.key;
    const sampleRate = this.sampleRate;

    // Built per-connection (inside openSocket) so a reconnect after a live
    // language switch picks up the current langCode.
    const buildWsUrl = (): string => {
      const params = new URLSearchParams({
        model:              model,
        language_code:      langCode,
        sample_rate:        String(sampleRate),
        input_audio_codec:  "pcm_s16le",
        high_vad_sensitivity: this.highVadSensitivity ? "true" : "false",
        vad_signals:        "true",    // emit speech_start / speech_end events
        flush_signal:       "true",    // allow manual buffer flush via flush() call
      });
      if (MODE_CAPABLE.has(model)) {
        params.set("mode", mode);
      }
      return `${WS_BASE}?${params.toString()}`;
    };

    let ws: WebSocket;
    let wsReady = false;
    let intentionallyClosed = false;
    let switchingLanguage = false; // set by setLanguage(); close handler reopens immediately, no ba
ckoff
    let reconnectAttempts = 0;
    let reconnectTimer: ReturnType<typeof setTimeout> | null = null;
    const MAX_RECONNECT = 10;

    const audioQueue: Buffer[] = [];
    const MAX_QUEUE_BYTES = 64_000; // ~2 s @ 16 kHz s16le — drop oldest chunks beyond this
    let queueTrimLogged   = false;  // throttle: log queue trim at most once per reconnect episode
    let speechEndAt       = 0;     // timestamp of last END_SPEECH event for latency tracking
    let speechEndReceived = false; // true once speech_end fires; reset on speech_start
    let lastRequestId: string | undefined; // latest request_id seen — attached to transcript/error
logs for Sarvam support

    // Closure variables for the onSpeechStart / onSpeechEnd callbacks. Declared here
    // (before openSocket) so the WS message handler — an async callback inside openSocket —
```

```
    // can reference them safely. Exposed on sttObj via Object.defineProperty getter/setter below.
    let _onSpeechStart: (() => void) | null = null;
    let _onSpeechEnd:   (() => void) | null = null;

    // ■■ Adaptive transcript coalescing ■■■
```

```
359       * caller is still formulating. Triggers COALESCE_INCOMPLETE_MS hold instead of
360       * the adaptive window so the caller can finish their thought.
361       *
362       * Patterns detected (Tamil-aware + generic):
363       *   • Single dangling connector (e.g. "and", "but", "■■■") — incomplete
364       *   • Single content word (name, number, noun, answer)      — COMPLETE (emits immediately)
365       *   • Ends with Tamil possessive suffix    — ■■■■■■■■, ■■■■■■■, ■■■■■, …
366       *   • Ends with Tamil case markers         — -■, -■■■■■, -■■■■■■, -■■, -■■■, …
367       *   • Ends with common connective words    — "and", "or", "but", "also", "with"
368       *   • All-caps abbreviation (e.g. "DTDC") — likely partial proper noun phrase
369       */
370      function looksIncomplete(text: string): boolean {
371        const t = text.trim();
372        const words = t.split(/\s+/);
373        if (words.length === 0) return false;
374        // Single word: only return true if it is a known dangling connector ("and", "but", "■■■" etc.
     )
375        // All other single words (names, numbers, nouns, greetings, yes/no) emit IMMEDIATELY.
376        if (words.length === 1) {
377          const w = words[0].toLowerCase().replace(/[■,!?.]+$/, "");
378          return DANGLING_SINGLE_CONNECTORS.has(w) || DANGLING_SINGLE_CONNECTORS.has(words[0]);
379        }
380        const last = words[words.length - 1];
381        // Tamil possessive / genitive suffixes → more to come
382        if (/[■■]■$|■■■■$|■■■■$/.test(last)) return true;
383        // Tamil dative, locative, instrumental markers
384        if (/■■■■$|■■■■■$|■■■■■■$|■■■$|■■■■$|■$/.test(last)) return true;
385        // Tamil conjunctive participial (connector) forms — ALWAYS signal more is coming:
386        //   -■■■■  : ■■■■■ (having come), ■■■■■■, ■■■■■■ (with)
387        //   -■■■■  : ■■■■■■ (having placed), ■■■■■■ (having let), ■■■■■…
388        //   -■■/-■■■■/-■■■■ : ■■■■■■ (having said), ■■■■■ (having done)
389        //   -■■    : ■■■■ (having gone), ■■■■
390        //   -■■■■ also covers vi■■u (■■■■■■■) and po■■u (■■■■■■■) naturally
391        if (/■■■■$|■■■■$|■■■■$|■■■■$|■■$|■■$/.test(last)) return true;
392        if (/■■$/.test(last) && last.length > 2) return true; // ■■■■, ■■■■ — skip 2-char words like "
     ■■■"
393        // Tamil relative-participial / adverbial forms
394        if (/■■■$|■■■$|■■■■$|■■■$/.test(last)) return true;
395        // 2-word utterance where the FIRST word ends with a case marker AND the second
396        // word is NOT a complete verb form → predicate is missing.
397        // "■■■■■■■ ■■■■■■" → incomplete ✓
398        // "■■■■■■■■■ ■■■■■■■■" → complete (second=verb) — should NOT fire
399        if (words.length === 2) {
400          const first = words[0];
401          const second = words[1];
402          // Tamil finite verb endings that make a 2-word sentence self-contained:
403          //   -■■■■/-■■■ (1ps past/future), -■■■■ (hortative), -■■■/-■■■■ (negative),
404          //   -■■■/-■■■■■ (equative), -■■■/-■■■■■■ (3p), -■■■■/-■■■■■ (neuter/future)
405          const secondIsFiniteVerb = /■■■$|■■■■$|■■■$|■■■■$|■■■$|■■■■$|■■■$|■■■$|■■■■■■$|■■■■$|■■■■■$|
     ■■■■$|■■■■■■$/.test(second);
406          if (!secondIsFiniteVerb && /■■■■$|■■■■■$|■■■■■■$|■$|■■■$|■■$/.test(first)) return true;
407        }
408        // Also check if FIRST word (not just last) is a connector/incomplete form.
409        // e.g. "■■■■■ ■■■■■■■" — "■■■■■" is conjunctive and signals more context.
410        const first = words[0];
411        if (/■■■■$|■■■■$|■■■■$|■■■■$/.test(first)) return true;
412        if (/■■$/.test(first) && first.length > 2) return true;
413        // Hindi/Hinglish possessives & case markers
414        if (/\b■■$|\b■■$|\b■■$|\b■■$|\b■■$|\b■■■$|\b■■$|\b■■$/u.test(last)) return true;
415        // Generic English connectives that signal more is coming
416        if (/^(and|or|but|also|with|for|of|to|the|a|an|in|on|at)$/i.test(last)) return true;
417        // English transitive / mental-state verbs that virtually always need a
418        // following object or complement. A bare utterance ending in one of these
419        // (no terminal punctuation, no object yet) means the caller is still
420        // mid-thought — e.g. "I want" → "I want to know the pricing…". Confirmed
421        // from call-1659: "I want" was flushed as a complete turn and produced a
422        // garbled follow-on ("I want a No good. worth pricing"). Skip this check
423        // when the text already ends with terminal punctuation — STT smart_format
424        // already judged it complete.
425        if (!/[.?!]$/.test(t)) {
426          const NEEDS_OBJECT_VERBS = new Set([
427            "want", "need", "like", "love", "hate", "have", "get", "know",
428            "think", "mean", "see", "hear", "feel", "believe", "wish", "hope",
429            "prefer", "use", "try", "check", "understand", "remember", "ask",
```

```
          "tell", "show", "give", "take", "make", "do", "call", "book",
          "send", "discuss", "explain", "mention", "consider", "suggest",
          "recommend",
        ]);
        if (NEEDS_OBJECT_VERBS.has(last.toLowerCase())) return true;
      }
      // All-caps token ≥ 2 chars (e.g. "DTDC", "KYC") — check both first and last word
      if (/^[A-Z]{2,}$/.test(last) || /^[A-Z]{2,}$/.test(first)) return true;
      // Tamil interrogative words — ALWAYS require a predicate; can never end a sentence.
      // ■■■■■ (which), ■■■■■ (what), ■■■■■/■■■■■■ (when), ■■■■■/■■■■■■ (where),
      // ■■■■■/■■■■■ (who), ■■■■■■■■ (how much), ■■■■■■■ (how), ■■■■ (why),
      // ■■■■ (which one), ■■■■■■■■ (for what), ■■■■■■■ (anything/which)
      if (/^(■■■■■|■■■■■|■■■■■|■■■■■■|■■■■■|■■■■■■|■■■■■|■■■■■|■■■■■■■■|■■■■■■■|■■■■|■■■■|■■■■■■■■|■■■■■■■|■■■■■■■|■■■■■■|■■■■■|■■■■■|■■■■■■)$/.test(last)) return true;
      // Hindi interrogatives: ■■■■■, ■■■, ■■■■■, ■■■■■, ■■■■, ■■■■■■, ■■■■■■
      if (/^(■■■■■|■■■|■■■■■|■■■■■|■■■■|■■■■■■|■■■■■■|■■■■■■|■■■■■■|■■■■■■)$/.test(last)) return true;
      return false;
    }

    let coalesceBuf  = "";
    let coalesceMeta: SttFinalMetadata | undefined;
    let coalesceLang: string | undefined;
    let coalesceTimer: ReturnType<typeof setTimeout> | null = null;

    const emitPartial = () => {
      const text = coalesceBuf.trim();
      if (!text) return;
      onTranscript(text, false, coalesceLang, {
        ...coalesceMeta,
        pendingPartial: true,
        speechEndReceived,
      });
    };

    const emitCoalesced = () => {
      coalesceTimer = null;
      if (!coalesceBuf) return;
      const text     = coalesceBuf;
      const lang     = coalesceLang;
      const segments = coalesceMergedSegments;
      const meta: SttFinalMetadata = {
        ...coalesceMeta,
        mergedSegments: segments,
        pauseCount:     Math.max(0, segments - 1),
      };
      coalesceBuf            = "";
      coalesceMeta           = undefined;
      coalesceLang           = undefined;
      coalesceMergedSegments = 0;
      transcriptArrivedAt    = 0;

      logger.info(
        { text: text.slice(0, 120), segments, holdMs: adaptiveHoldMs(), pauseHistory: pauseHistory.slice(), detectedLang: lang },
        "Sarvam STT: emitting adaptive-coalesced transcript",
      );
      onTranscript(text, true, lang, meta);
    };

    // ■■ WebSocket health monitoring ■■■■■■■■
```

```
    const WS_DEAD_MS     = 3_000;

    const sendChunk = (chunk: Buffer) => {
      if (!wsReady || ws.readyState !== WebSocket.OPEN) return;
      const now = Date.now();
      // Resuming after a send gap (AMD confirmation, pre-greeting, carrier DTX/stream
      // pause — the feed itself is NOT gated during agent playback):
      // grant Sarvam a fresh WS_DEAD_MS window before the zombie check can fire. Without
      // this, a stale lastMsgAt from before the gap would trip a false positive on the
      // very first chunk after playback and terminate a perfectly healthy socket.
      if (now - lastAudioSentAt > WS_DEAD_MS) lastMsgAt = now;
      ws.send(
        JSON.stringify({
          audio: {
            data:        chunk.toString("base64"),
            encoding:    "audio/wav",   // enum must be "audio/wav"; actual codec is set via URL par
am
            sample_rate: sampleRate,
          },
        }),
      );
      lastAudioSentAt = now; // proves we are ACTIVELY streaming — gates the zombie detector
    };

    const openSocket = () => {
      if (intentionallyClosed) return;
      wsReady = false;

      ws = new WebSocket(buildWsUrl(), {
        headers: { "api-subscription-key": key },
      });

      ws.on("open", () => {
        wsReady = true;
        reconnectAttempts = 0;
        deadFired         = false; // fresh socket — re-arm the zombie detector
        lastMsgAt         = Date.now(); // start silence window from now, not dead socket's last fra
me
        queueTrimLogged   = false; // re-arm queue-trim log for this reconnect episode
        // Reset VAD state: a stale speechEndReceived=true would send post-reconnect
        // transcripts down the 30ms fast-flush path before any fresh speech_end fires.
        inSpeechSegment   = false;
        speechEndReceived = false;
        logger.info(
          { langCode, model, mode },
          "Sarvam streaming STT: WebSocket opened",
        );
        // Flush any audio that arrived before the socket was ready.
        for (const chunk of audioQueue.splice(0)) sendChunk(chunk);
      });

      ws.on("message", (raw: Buffer | string) => {
        let msg: SarvamMsg;
        try {
          msg = JSON.parse(raw.toString()) as SarvamMsg;
        } catch {
          return; // ignore non-JSON frames
        }

        // VAD signals — official type values: "speech_start" / "speech_end"
        // (legacy guessed values "events" + signal_type also handled for safety)
        lastMsgAt = Date.now();
        deadFired = false; // any inbound frame proves the socket is alive — re-arm the detector

        // Keep the latest request_id (Sarvam docs: support/debugging requests are
        // resolved fastest when the affected call's request_id can be quoted).
        const _reqId = msg.request_id ?? msg.data?.request_id;
        if (_reqId) lastRequestId = _reqId;
        if (msg.type === "speech_start" || (msg.type === "events" && msg.data?.signal_type === "STAR
T_SPEECH")) {
          inSpeechSegment    = true;
          speechEndReceived  = false; // caller is speaking again; clear the speech_end flag
          _onSpeechStart?.();
          logger.debug({ langCode }, "Sarvam STT: speech start");
```

```
          // If we are holding a transcript and the user resumed speaking, cancel
          // the emit timer, record the pause length for adaptive calibration, and
          // wait for the next transcript to append before emitting.
          if (coalesceTimer && coalesceBuf) {
            clearTimeout(coalesceTimer);
            coalesceTimer = null;
            if (transcriptArrivedAt > 0) {
              const pauseMs = Date.now() - transcriptArrivedAt;
              if (pauseMs > 40 && pauseMs < 1200) { // filter: ignore sub-40ms noise & >1.2s stale gaps
                pauseHistory.push(pauseMs);
                if (pauseHistory.length > PAUSE_HISTORY_MAX) pauseHistory.shift();
              }
              transcriptArrivedAt = 0;
            }
            logger.debug(
              { langCode, pauseHistory: pauseHistory.slice(), nextHoldMs: adaptiveHoldMs() },
              "Sarvam STT: coalesce hold cancelled — speech resumed, pause recorded",
            );
          }
          return;
        }
        if (msg.type === "speech_end" || (msg.type === "events" && msg.data?.signal_type === "END_SPEECH")) {
          inSpeechSegment   = false;
          speechEndReceived = true;
          speechEndAt       = Date.now();
          logger.debug({ langCode }, "Sarvam STT: speech end");
          // Fast-path when transcript is ALREADY buffered (multi-segment coalescing):
          // caller has stopped talking — shorten any running timer.
          // Universal thinking-pause rule (language-agnostic):
          //   • CLEARLY COMPLETE  → 30 ms fast-flush
          //       – Terminal punctuation (.?!■) — STT itself declared it done
          //       – Single known-affirmation word ("yes", "■■■", "■■■", …)
          //   • EVERYTHING ELSE   → max(adaptiveHoldMs(), 700 ms)
          //       Covers ALL mid-sentence fragments in ALL languages without
          //       enumerating grammar patterns. "5 lakhs", "■■■■■■■ ■■■■■■",
          //       "mujhe chahiye", "I want", Telugu/Kannada partials, etc.
          //       all wait 700 ms, which is enough to survive a normal human
          //       thinking pause (~1 s) while the STT silence window is shorter.
          if (coalesceBuf && coalesceTimer) {
            clearTimeout(coalesceTimer);
            // speech_end received: caller has stopped speaking — flush (UI-configured)
            coalesceTimer = setTimeout(emitCoalesced, SPEECH_END_FLUSH_MS);
            emitPartial();
            logger.debug(
              { langCode, buf: coalesceBuf.slice(0, 60), flushMs: SPEECH_END_FLUSH_MS },
              "Sarvam STT: speech_end received — flushing coalesce buffer",
            );
          }
          _onSpeechEnd?.();
          return;
        }

        // Final transcript — official type: "transcript"; legacy: "data"
        if (msg.type === "transcript" || msg.type === "data") {
          inSpeechSegment = false; // segment complete
          const text = stripDiarizationLabels(msg.data?.transcript ?? "");
          // Capture the language Sarvam identified for this segment (saaras:v3 LID).
          // Last seen language_code wins across coalesced segments; reset in emitCoalesced().
          //
          // IMPORTANT: only forward the reported code when it DIFFERS from the
          // init language (langCode). When Sarvam is initialised with
          // language_code=en-IN it echoes "en-IN" on every response regardless
          // of what the caller actually spoke — this is NOT real LID; it is just
          // the init-language echoed back. Forwarding that echo would suppress
          // detectScriptLanguage() in the pipeline: the pipeline short-circuits on
          // a non-null sttLang and never reads Unicode script blocks in the
          // transcript. Leaving coalesceLang undefined on "same-as-init" responses
          // lets detectScriptLanguage() run and catch actual Indic-script characters
          // (Tamil, Hindi, etc.) in the transcript text.
          const reportedLang = msg.data?.language_code;
          if (reportedLang && reportedLang !== langCode) {
            coalesceLang = reportedLang;
```

```
645           }
646           if (!text) return;
647           const meta: SttFinalMetadata = {
648             endDetectToFinalMs: speechEndAt > 0 ? Date.now() - speechEndAt : undefined,
649             requestId: lastRequestId,
650           };
651           logger.info(
652             { text: text.slice(0, 80), model, mode, endDetectToFinalMs: meta.endDetectToFinalMs, req
    uestId: lastRequestId },
653             "Sarvam STT: final transcript (buffering for adaptive coalesce)",
654           );
655           speechEndAt = 0;
656           // Append to coalesce buffer (may already have a prior partial).
657           coalesceBuf            = coalesceBuf ? coalesceBuf + " " + text : text;
658           coalesceMeta           = meta;         // keep latest latency measurement
659           coalesceMergedSegments++;
660           transcriptArrivedAt    = Date.now();   // start of the hold window
661           // (Re-)arm the emit timer.
662           // Timer selection after each transcript segment:
663           // • speech_end already received → SPEECH_END_FLUSH_MS (UI, default 30).
664           // • incomplete mid-thought → COALESCE_INCOMPLETE_MS (UI, default 220).
665           // • complete transcript, speech_end not yet → SPEECH_END_FLUSH_MS.
666           if (coalesceTimer) clearTimeout(coalesceTimer);
667           const incomplete = looksIncomplete(coalesceBuf);
668           const holdMs = speechEndReceived
669             ? SPEECH_END_FLUSH_MS
670             : (incomplete ? COALESCE_INCOMPLETE_MS : SPEECH_END_FLUSH_MS);
671           logger.debug(
672             { langCode, buf: coalesceBuf.slice(0, 60), holdMs, speechEndReceived },
673             "Sarvam STT: scheduling coalesce emit",
674           );
675           coalesceTimer = setTimeout(emitCoalesced, holdMs);
676           emitPartial();
677           return;
678         }
679
680         if (msg.type === "error") {
681           logger.error(
682             { msg: msg.data?.message, requestId: msg.request_id ?? msg.data?.request_id ?? lastReque
    stId, langCode },
683             "Sarvam STT: server error",
684           );
685         }
686       });
687
688       ws.on("close", (code, reason) => {
689         wsReady = false;
690
691         if (intentionallyClosed) {
692           onClose?.();
693           return;
694         }
695
696         // Live language switch: we closed the socket ourselves — reopen
697         // immediately with the new langCode. Not an error; no backoff and no
698         // reconnectAttempts++ (audio buffered in audioQueue meanwhile).
699         if (switchingLanguage) {
700           switchingLanguage = false;
701           logger.info({ langCode, model }, "Sarvam STT: reopening WebSocket after language switch");
702           openSocket();
703           return;
704         }
705
706         // Auth / quota errors: do not retry.
707         if (code >= 4000 && code < 5000) {
708           logger.error(
709             { code, reason: reason.toString() },
710             "Sarvam STT: auth/quota close — not retrying",
711           );
712           onClose?.();
713           return;
714         }
715
```

```
        if (reconnectAttempts >= MAX_RECONNECT) {
          logger.error({ code, reconnectAttempts }, "Sarvam STT: max reconnects reached");
          onClose?.();
          return;
        }

        // Exponential backoff: 500ms, 1s, 2s, 4s … capped at 8s.
        const delay = Math.min(500 * 2 ** reconnectAttempts, 8000);
        reconnectAttempts++;
        logger.warn(
          { code, delay, reconnectAttempts },
          "Sarvam STT: WebSocket closed — reconnecting",
        );
        if (reconnectTimer) clearTimeout(reconnectTimer);
        reconnectTimer = setTimeout(() => {
          reconnectTimer = null;
          openSocket();
        }, delay);
      });

      ws.on("error", (err) => {
        logger.error({ err }, "Sarvam STT: WebSocket error");
      });
    };

    openSocket();

    const sttObj: import("../../types.js").StreamSTT = {
      onConnectionDead: null,

      sendAudio(chunk: Buffer): void {
        if (!wsReady || ws.readyState !== WebSocket.OPEN) {
          audioQueue.push(chunk);
          // Cap the queue at ~2 s of audio — drop oldest chunks so that on reconnect
          // we don't flood Sarvam with stale audio and produce phantom transcripts.
          let total = audioQueue.reduce((a, b) => a + b.length, 0);
          if (total > MAX_QUEUE_BYTES) {
            let dropped = 0;
            while (total > MAX_QUEUE_BYTES && audioQueue.length > 1) {
              const len = audioQueue.shift()!.length;
              dropped  += len;
              total    -= len;
            }
            if (!queueTrimLogged) {
              queueTrimLogged = true;
              logger.debug({ droppedBytes: dropped }, "Sarvam STT: audio queue trimmed — dropped sta
le chunks during reconnect backoff");
            }
          }
        } else {
          sendChunk(chunk);
        }
      },
      // INVARIANT: the live pipeline never calls flush() — Sarvam's server VAD owns
      // end-of-speech (see the DO-NOT-GATE feed comment in pipeline.ts). Adding a
      // pipeline call site would reintroduce client-side endpointing racing Sarvam
      // (the calls-772→1320 regression class). flush() exists for teardown/tooling.
      flush(): void {
        // Emit any held coalesced transcript immediately (don't wait for the timer).
        // emitCoalesced() is a no-op on an empty buffer, so always call it — this
        // handles the edge case where coalesceBuf is non-empty but coalesceTimer is null.
        if (coalesceTimer) clearTimeout(coalesceTimer);
        emitCoalesced();
        // Ask Sarvam to immediately finalize its internal audio buffer.
        if (!wsReady || ws.readyState !== WebSocket.OPEN) return;
        const emptyData = Buffer.alloc(0);
        ws.send(
          JSON.stringify({
            audio: {
              data:        emptyData.toString("base64"),
              encoding:    "audio/wav",
              sample_rate: sampleRate,
            },
            flush: true,
```

```
          }),
        );
        logger.debug({ langCode }, "Sarvam STT: flush signal sent");
      },
      setLanguage(lang: string): void {
        const next = normaliseLang(lang, model);
        if (next === langCode) return;           // already on it — no-op (loop guard)
        if (intentionallyClosed) return;         // stream already torn down
        const prev = langCode;
        langCode = next;
        mode = resolveSarvamSttMode(langCode, explicitMode);
        logger.info(
          { from: prev, to: next, mode, model },
          "Sarvam STT: live language switch — reconnecting stream with new language hint",
        );
        // Drain any transcript held in the OLD language before switching.
        if (coalesceTimer) { clearTimeout(coalesceTimer); emitCoalesced(); }
        if (ws && (ws.readyState === WebSocket.OPEN || ws.readyState === WebSocket.CONNECTING)) {
          switchingLanguage = true;              // close handler reopens immediately with new langC
ode
          try { ws.close(1000); } catch {
            switchingLanguage = false;
            try { ws.terminate(); } catch { /* noop */ }
            openSocket();
          }
        }
        // else: socket is between reconnects — the pending reconnect timer's
        // openSocket() will pick up the new langCode via buildWsUrl().
      },
      close(): void {
        // Drain the coalesce buffer before closing so no transcript is lost.
        if (coalesceTimer) { clearTimeout(coalesceTimer); emitCoalesced(); }
        intentionallyClosed = true;
        if (reconnectTimer) {
          clearTimeout(reconnectTimer);
          reconnectTimer = null;
        }
        clearInterval(heartbeatInterval);
        if (
          ws.readyState === WebSocket.OPEN ||
          ws.readyState === WebSocket.CONNECTING
        ) {
          ws.close(1000);
        }
      },
      getPendingTranscript(): string {
        return coalesceBuf.trim();
      },
    };

    // Expose _onSpeechStart / _onSpeechEnd via getter/setter so the WS handler (which
    // closes over these variables, declared before openSocket) stays in sync with whatever
    // callback the caller assigns after sttObj is returned.
    Object.defineProperty(sttObj, "onSpeechStart", {
      get(): (() => void) | null { return _onSpeechStart; },
      set(cb: (() => void) | null): void { _onSpeechStart = cb; },
      configurable: true,
      enumerable: true,
    });
    Object.defineProperty(sttObj, "onSpeechEnd", {
      get(): (() => void) | null { return _onSpeechEnd; },
      set(cb: (() => void) | null): void { _onSpeechEnd = cb; },
      configurable: true,
      enumerable: true,
    });

    // Sarvam closes the streaming WS after ~60 s of inactivity (documented
    // limit). The pipeline gates caller frames during agent playback, so a long
    // agent monologue / hold can starve the socket into an idle close + a
    // pointless reconnect cycle. After 15 s with nothing sent, ship a short
    // near-silent PCM frame as a keep-alive (official guidance). Sent RAW — not
    // via sendChunk — so it never updates lastAudioSentAt/lastMsgAt: keep-alives
    // must neither arm the zombie detector nor count as "actively streaming".
    const KEEPALIVE_IDLE_MS = 15_000;
```

```
    const KEEPALIVE_FRAME   = Buffer.alloc(Math.floor(sampleRate * 0.2) * 2); // 200 ms silence, s16le mono
    let   lastKeepAliveAt   = 0;

    // Start the heartbeat AFTER sttObj is created so the interval can reference it.
    const heartbeatInterval = setInterval(() => {
      if (intentionallyClosed) return;                    // torn down
      // ■■ Idle keep-alive (only OUTSIDE speech segments — real audio covers the rest) ■■
      const nowMs = Date.now();
      if (
        wsReady && ws.readyState === WebSocket.OPEN && !inSpeechSegment &&
        nowMs - lastAudioSentAt > KEEPALIVE_IDLE_MS &&
        nowMs - lastKeepAliveAt > KEEPALIVE_IDLE_MS
      ) {
        lastKeepAliveAt = nowMs;
        try {
          ws.send(JSON.stringify({
            audio: {
              data:        KEEPALIVE_FRAME.toString("base64"),
              encoding:    "audio/wav",
              sample_rate: sampleRate,
            },
          }));
          logger.debug({ langCode }, "Sarvam STT: idle keep-alive frame sent (60s inactivity guard)");
        } catch { /* socket racing closed — the close handler's reconnect path covers it */ }
      }
      if (deadFired) return;                              // zombie already handled this episode
      if (!inSpeechSegment) return;                       // zombie check only meaningful mid-utterance
      if (ws.readyState !== WebSocket.OPEN) return;       // CONNECTING/CLOSING handled elsewhere
      // Suppress false positives: during genuine send gaps (AMD confirmation phase,
      // pre-greeting window, carrier DTX/stream pauses) we are NOT sending audio and
      // Sarvam is legitimately silent. Only suspect death while actively streaming.
      if (Date.now() - lastAudioSentAt > WS_DEAD_MS) return;
      const silenceMs = Date.now() - lastMsgAt;
      if (silenceMs > WS_DEAD_MS) {
        deadFired = true; // fire-once latch: reset by any inbound message or a fresh socket
        logger.warn(
          { langCode, silenceMs },
          "Sarvam STT: WS silent while streaming audio — terminating zombie socket to force reconnect",
        );
        try { sttObj.onConnectionDead?.(); } catch { /* optional external consumer hook */ }
        // Self-heal: terminate the dead socket so ws.on("close") runs the existing
        // exponential-backoff reconnect (terminate → 1006, which is not an auth code).
        try { ws.terminate(); } catch { try { ws.close(); } catch { /* noop */ } }
      }
    }, HEARTBEAT_MS);

    return sttObj;
  }
}
```